

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Специальность ИИ-26(2)

Выполнил
А. В. Семёнов,
студент группы ИИ-26

Проверила
Андренко К.В.

Брест 2026

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Задание 1. Обучение свёрточной нейронной сети

Построить простую свёрточную нейронную сеть средствами PyTorch, обучить её на заданной выборке, оценить качество на тестовых данных и построить график изменения ошибки.

Вариант 16: выборка MNIST, размер исходного изображения 28×28 пикселей, оптимизатор RMSprop.

Выполнение

MNIST содержит полутоновые изображения рукописных цифр от 0 до 9. Данные загружаются через torchvision.datasets.MNIST: 60 000 обучающих и 10 000 тестовых изображений. Из обучающей части выделено 5 000 изображений для валидации; веса обновляются на оставшихся 55 000.

Разбиение выполнено случайно с seed = 16. Test не участвует в обучении и выборе эпохи. Используется одна модель, без предобученных весов, аугментации и ансамблей. ToTensor переводит яркости в [0; 1], Normalize со средним 0,5 и стандартным отклонением 0,5 — в [-1; 1]. Размер 28×28 сохраняется.

Архитектура модели

Слой	Выход без размера пакета	Параметры
Вход	1×28×28	0
Conv2d, 1→16, 3×3, padding=1	16×28×28	160
ReLU и MaxPool2d, 2×2	16×14×14	0
Conv2d, 16→32, 3×3, padding=1	32×14×14	4 640
ReLU и MaxPool2d, 2×2	32×7×7	0
Flatten	1 568	0
Linear, 1 568→64 и ReLU	64	100 416
Linear, 64→10	10	650
Всего		105 866

Свёртки выделяют локальные признаки штрихов, ReLU задаёт нелинейность, а MaxPool уменьшает пространственный размер карт признаков. Два полносвязных слоя преобразуют 1 568 признаков в десять логитов. Номер максимального логита принимается за распознанную цифру.

Код программы

Модель определена в lab1.py. Обучение, оценка и распознавание объединены в одном исходном файле.

```
from torch import nn

class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, x):
        # CrossEntropyLoss получает логиты, поэтому Softmax здесь не нужен.
        return self.classifier(self.features(x))
```

Обучение модели

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.RMSprop(
    model.parameters(), lr=0.001, alpha=0.99,
    eps=1e-8, momentum=0, weight_decay=0)

model.train()
for images, targets in train_loader:
    images, targets = images.to(device), targets.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(images)
    loss = criterion(logits, targets)
    loss.backward()
    optimizer.step()
```

CrossEntropyLoss получает логиты и целочисленные метки 0–9. Для одного изображения $L = -\log(\exp(z_i) / \sum_j \exp(z_j))$. При обучении используется среднее значение по пакету. Softmax перед этой функцией потерь не применяется.

RMSprop хранит скользящее среднее квадратов градиента: $v_t = 0,99v_{t-1} + 0,01g_t^2$. Параметры обновляются по правилу $\theta_t = \theta_{t-1} - 0,001g_t / (\sqrt{v_t} + 10^{-8})$. Все операции выполняются поэлементно.

После каждой эпохи сеть переводится в `model.eval()`, и в режиме без вычисления градиентов оценивается validation loss. Сохраняются веса с её минимальным значением. Фиксировано 12 эпох; scheduler и ранняя остановка не используются.

Спецификация ввода

Команда запуска из папки с lab1.py:

```
python lab1.py
```

При обычном запуске выполняется вся работа: 12 эпох, пакет обучения 128, оценки 512, $lr = 0,001$, $seed = 16$. Эксперимент выполнен с Python 3.12.14, PyTorch 2.7.1+cu126 и torchvision 0.22.1+cu126 на NVIDIA GeForce RTX 4060. Начальные состояния генераторов зафиксированы; включены детерминированные алгоритмы.

Результаты обучения

Эпоха	Train loss	Val loss	Train, %	Val, %
1	0,2407	0,0913	92,54	97,28
2	0,0703	0,0679	97,76	97,98
3	0,0500	0,0601	98,50	98,32
4	0,0398	0,0523	98,76	98,54
5	0,0314	0,0487	99,04	98,56
6	0,0253	0,0705	99,19	98,08
7	0,0207	0,0480	99,30	98,70
8	0,0173	0,0612	99,49	98,34
9	0,0137	0,0605	99,57	98,50
10	0,0125	0,0471	99,61	98,80
11	0,0106	0,0560	99,66	98,82
12	0,0084	0,0583	99,73	98,54

Train — средние показатели по пакетам во время обновления весов; Val — оценка модели в конце эпохи. Поэтому эти значения получены в разных режимах измерения.

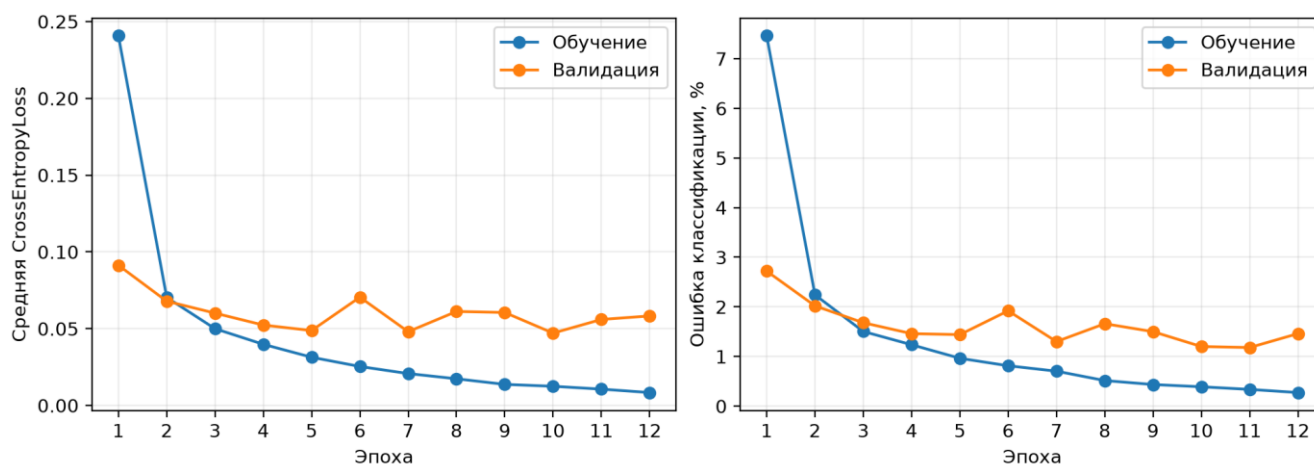


Рисунок 1 — Изменение функции потерь и доли ошибок по эпохам

По минимальной validation loss выбрана эпоха 10: $loss = 0,0471$, $точность = 98,80\%$. После загрузки сохранённых весов на test получено 9910 верных ответов из 10000, $точность 99,10\%$, $ошибка 0,90\%$, $CrossEntropyLoss = 0,0305$.

Проверка на тестовой выборке

Тестирование выполнено на всех 10 000 изображениях. Модель допустила 90 ошибок. В матрице ошибок строка соответствует истинной цифре, столбец — ответу сети; правильные ответы расположены на главной диагонали.

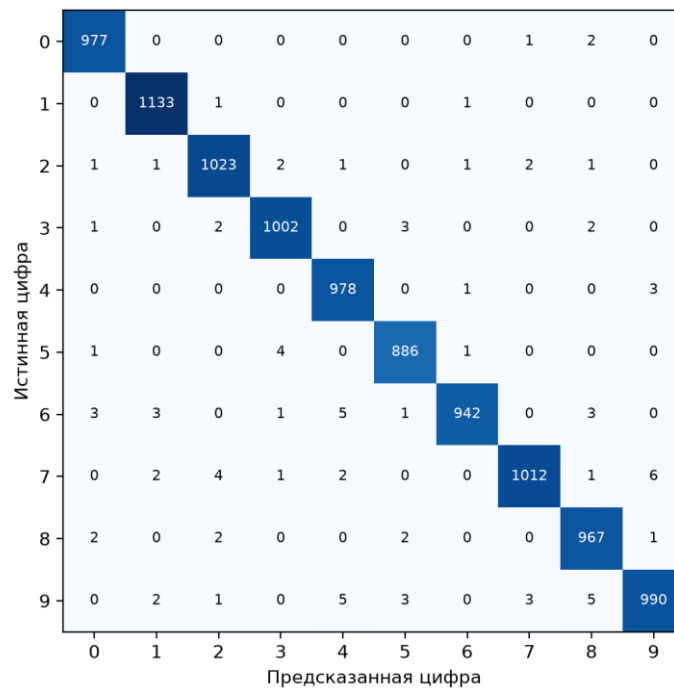


Рисунок 2 — Матрица ошибок на MNIST test

Первые ошибки в порядке тестовой выборки

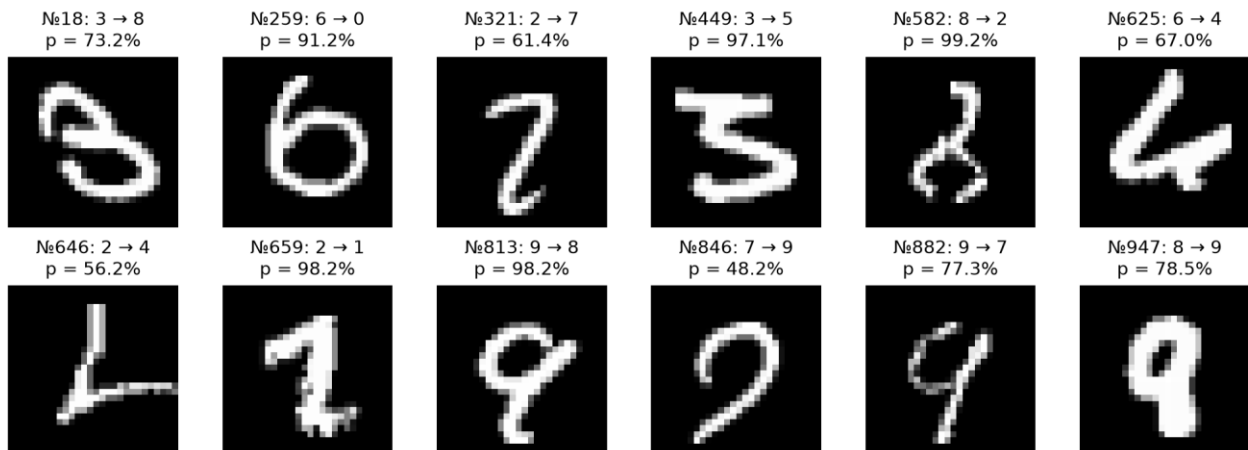


Рисунок 3 — Первые 12 ошибок по порядку изображений в test

На рисунке 3 указаны индекс изображения, истинная цифра, ответ сети и оценка Softmax выбранного класса. Ошибки показаны без отбора по внешнему виду. Высокая оценка Softmax может сопровождать неверный ответ, поэтому она не является гарантией правильного распознавания.

Индивидуальные ответы всех 10 000 изображений сохранены в test_predictions.csv. Матрица в confusion_matrix.csv позволяет проверить общее число ответов и количество ошибок независимо от графика.

К 12-й эпохе train loss снизилась до 0,0084, но validation loss выросла с 0,0471 на 10-й эпохе до 0,0583. Это указывает на начало переобучения; для итогового теста использованы более ранние сохранённые веса.

Задание 2. Сравнение с опубликованными результатами

Ознакомиться с результатами state-of-the-art для MNIST и сопоставить их с результатом построенной сети.

Выполнение

Для сравнения рассмотрены первичные публикации о высокоточных моделях MNIST. Указанные ниже показатели относятся к экспериментам авторов; они не означают, что все модели обучались в одинаковых условиях.

Модель и источник	Точность, %	Ошибка, %
Построенная DigitCNN	99,10	0,90
MCDNN, Cireşan и др., 2012 [3]	99,77	0,23
Ансамбль трёх CNN, An и др., 2020 [4]	99,87	0,13
Двухуровневый ансамбль, An и др., 2020 [4]	99,91	0,09

В работе Cireşan и соавторов MCDNN объединяет 35 сетей, обученных с разными вариантами предварительной обработки изображений. Авторы получили 23 ошибки на тестовой выборке MNIST [3].

An и соавторы используют свёртки с ядрами 3×3, 5×5 и 7×7, нормализацию по пакету, повороты и сдвиги обучающих изображений. Для ансамбля из трёх сетей опубликован лучший результат 99,87%, для двухуровневого ансамбля из девяти сетей — 99,91% [4].

Показатель 99,91% соответствует лучшему найденному сочетанию моделей. В публикации контрольные точки и ансамбли отбираются с учётом test accuracy. В данной работе эпоха выбрана по отдельной валидационной части, поэтому процедуры оценки также различаются.

Вывод по сравнению

Ошибка построенной сети составляет 0,90%. Разница с результатом 0,09% равна 0,81 процентного пункта, или 81 дополнительным ошибкам на 10 000 изображениях. Это сравнение опубликованных показателей, а не измерение влияния одного только оптимизатора.

Простая сеть с 105 866 параметрами решает задачу классификации MNIST, но уступает исследовательским ансамблям. Возможные улучшения — умеренные повороты и сдвиги при обучении, регуляризация и снижение шага обучения. Проверять такие изменения следует на validation, сохраняя test для итоговой оценки.

Приведены известные опубликованные ориентиры высокого качества, а не утверждение об абсолютном рекорде на дату выполнения работы. Результат лабораторной относится к одному запуску с фиксированным seed.

Задание 3. Распознавание выбранного изображения

Реализовать выбор произвольного изображения, подачу его на обученную сеть и визуализацию результата.

Выполнение

В lab1.py реализованы два способа выбора: путь через параметр `--image` и стандартный диалог выбора файла. Загружаются сохранённые веса `model.pt`, сеть переводится в режим оценки. Программа обрабатывает изображение одной цифры; распознавание строк из нескольких цифр в эту задачу не входит.

В режиме `auto` изображение переводится в оттенки серого. По яркости краёв определяется фон; тёмная цифра на светлом фоне инвертируется. После удаления пустых полей цифра вписывается в квадрат 20×20 с сохранением пропорций, помещается на чёрный холст 28×28 и центрируется по массе яркости. Сдвиг ограничен, чтобы штрихи не обрезались. Затем применяется та же нормализация, что при обучении.

Код распознавания

```
prepared = prepare_image(args.image, args.mode)
model, checkpoint = load_model(args.checkpoint)
with torch.inference_mode():
    x = image_transform()(prepared).unsqueeze(0)
    probabilities = model(x).softmax(1)[0].numpy()
    prediction = int(probabilities.argmax())
```

Спецификация ввода

```
python lab1.py predict --image examples/digit_7.png --show
```

При обычном запуске после обучения автоматически открываются все графики и диалог выбора своей цифры. Отмена пропускает этот шаг. Команда `predict` выполняет только распознавание; без `--image` она открывает диалог. Режим `mnist` сохраняет исходное расположение готовой цифры 28×28 , `dark` и `light` задают цвет цифры.

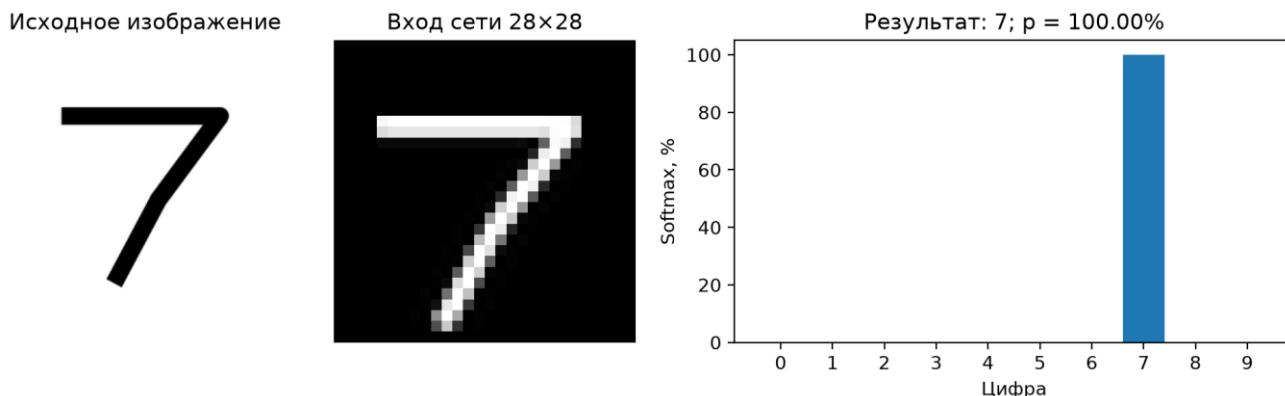


Рисунок 4 — Распознавание отдельного изображения цифры

Для демонстрации создан отдельный файл `digit_7.png` размером 280×280 , содержащий программно нарисованную цифру 7. Он не входит в MNIST. Модель выдала 7, оценка Softmax выбранного класса — 100,00%. Этот пример иллюстрирует обработку внешнего файла и не включается в расчёт `test accuracy`.

Спецификация вывода

Программа сама создаёт папки data, results и examples рядом с lab1.py. В results сохраняются веса, метрики и графики. Распознавание выводит цифру и оценки классов в консоль, PNG и JSON. Для повторного просмотра одной цифры без обучения используется команда predict.

```
python lab1.py predict --image examples/mnist_test_0.png --mode mnist --show
```

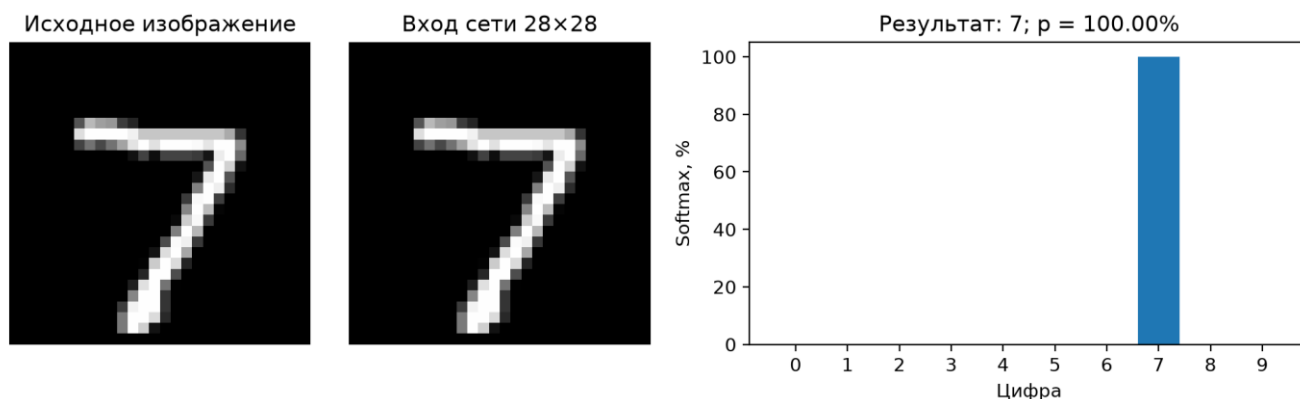


Рисунок 5 — Контрольный пример из тестовой выборки

Для изображения MNIST с индексом 0 истинная цифра — 7. Ответ программы — 7, оценка Softmax — 100,00%. В режиме mnist вход совпадает с изображением, использованным при тестировании.

Вывод

Выполнено конструирование и обучение свёрточного классификатора средствами PyTorch для варианта 16. Использованы MNIST и RMSprop. На тестовой выборке из 10 000 изображений получена точность 99,10% при 90 ошибках. Построены графики изменения потерь и ошибки классификации, выполнено сравнение с опубликованными результатами и реализовано распознавание выбранного файла.

Модель рассчитана на отдельные рукописные цифры, близкие по виду к MNIST. Буквы, фотографии и сложный фон могут приводить к неверному ответу даже при высокой оценке Softmax.